



Modélisation et analyse qualitative de systèmes

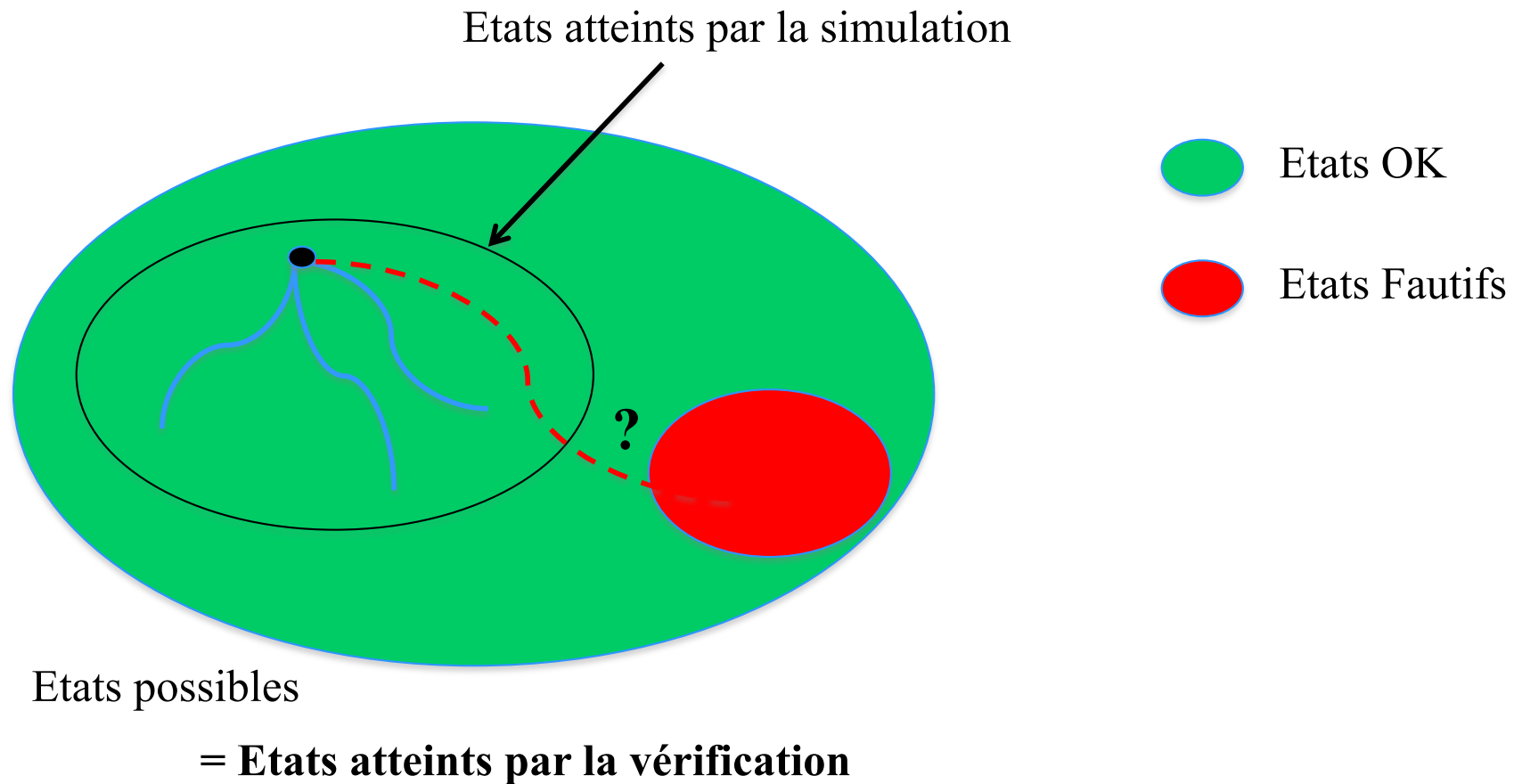
Modélisation d'application en PROMELA
et utilisation de l'outil SPIN pour son analyse

`http://spinroot.com/spin/whatispin.html`

spin et iSpin

- `spin` est un outil permettant la simulation et la vérification d'algorithmes répartis
- `iSpin` est son interface graphique
- Les algorithmes sont décrits dans le langage `promela`
- `promela` permet la représentation de
 - Processus concurrents
 - Mémoire partagée
 - Communication par message
 - Synchronisation
- La simulation et la vérification de systèmes concrets (écrits en C) ont été réalisées avec `spin`.

Pourquoi vérifier ?



promela

- Éléments constitutifs
 - Des variables (globales et/ou locales)
 - Des processus concurrents strictement asynchrones
 - Des canaux de communication (globaux et/ou locaux) de type FIFO borné
 - Variables
 - Types de base
 - `bit, bool, byte, short, int`
 - `int i; short s = 0;`
 - Tableaux statiques
 - `byte tab[10]; /* indices de 0 à 9 */`
- Promela ne distingue pas `tab` de `tab[0]`.
- l'instruction `c!tab` émet sur le canal `c` un message ne contenant que la valeur `tab[0]`.

promela : définition des processus

```
proctype <nom_proctype>(<paramètres formels>) {  
    instructions  
}
```

- *définit* le comportement type des processus `nom_proctype`
- un processus peut avoir des paramètres (types de base ou canaux)
- le processus particulier `init` (s'il existe) est lancé au démarrage de l'application
- Un processus devient *actif*
 - Lorsqu'il est instancié par l'instruction `run` (souvent dans `init`)
`run <nom_proctype>(<paramètres effectifs>)`
 - Lors du démarrage de l'application s'il est déclaré `active`
`active proctype <nom_proctype>() {`
 ...
`}`

promela : instructions des processus

- L'exécution d'une instruction est atomique
- Instructions :
 - condition (instruction bloquante si la condition n'est pas satisfaite)
 - affectation (toujours exécutable)
les expressions d'affectation et les conditions booléennes utilisent la syntaxe du C
 - émission sur un canal de communication !
(instruction bloquante si le canal est saturé, ou perte du message émis)
 - réception sur un canal de communication ?
(instruction bloquante si le canal est vide)
- Deux séparateurs d'instructions possibles : ; et ->
- Structures de contrôle conditionnelle (if) et répétitive (do)

Premier programme en promela

```
byte state = 1;
proctype A() {
    byte tmp;
    (state==1) -> tmp = state; tmp = tmp+1; state = tmp;
}
proctype B() {
    byte tmp;
    (state==1) -> tmp = state; tmp = tmp-1; state = tmp;
}

init {
    run A();
    run B();
}
```

variable globale

variable locale

condition

affectation

Exécution du programme

```
init {  
    run A();  
    run B()  
}
```

A est lancé avant B. A commence à s'exécuter avant le lancement de B. **Ou non.**

```
proctype A() {  
    byte tmp;  
    (state==1) -> tmp = state; tmp = tmp+1; state = tmp  
}
```

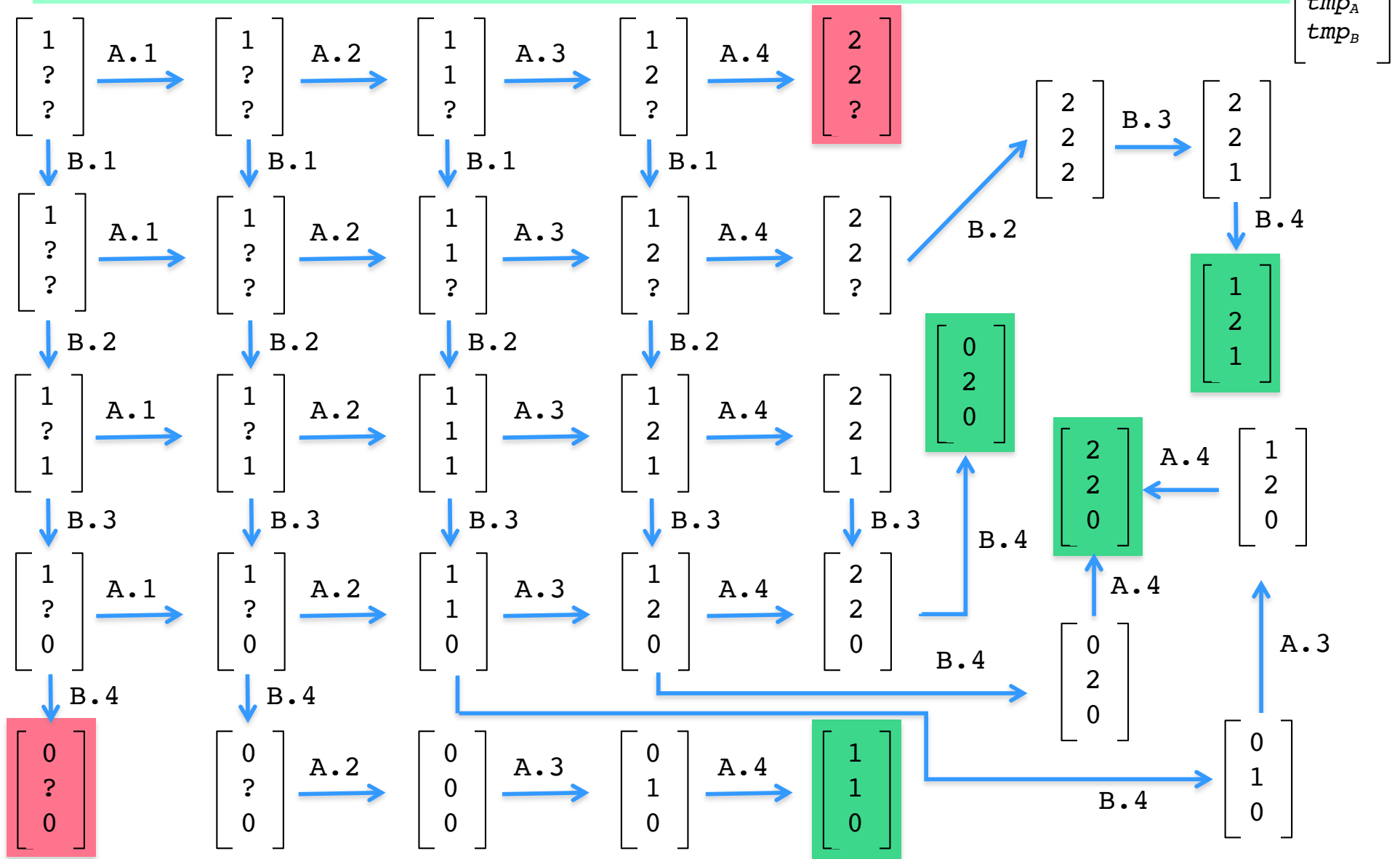
```
proctype B() {  
    byte tmp;  
    (state==1) -> tmp = state; tmp = tmp-1; state = tmp  
}
```

L'exécution d'un processus n'est pas atomique

La variable `state` peut prendre la valeur 0, 1 ou 2 à la fin de l'exécution

Un des processus peut se trouver définitivement bloqué

Graphe des états



Instructions atomiques (test and set)

```
byte state = 1;
```

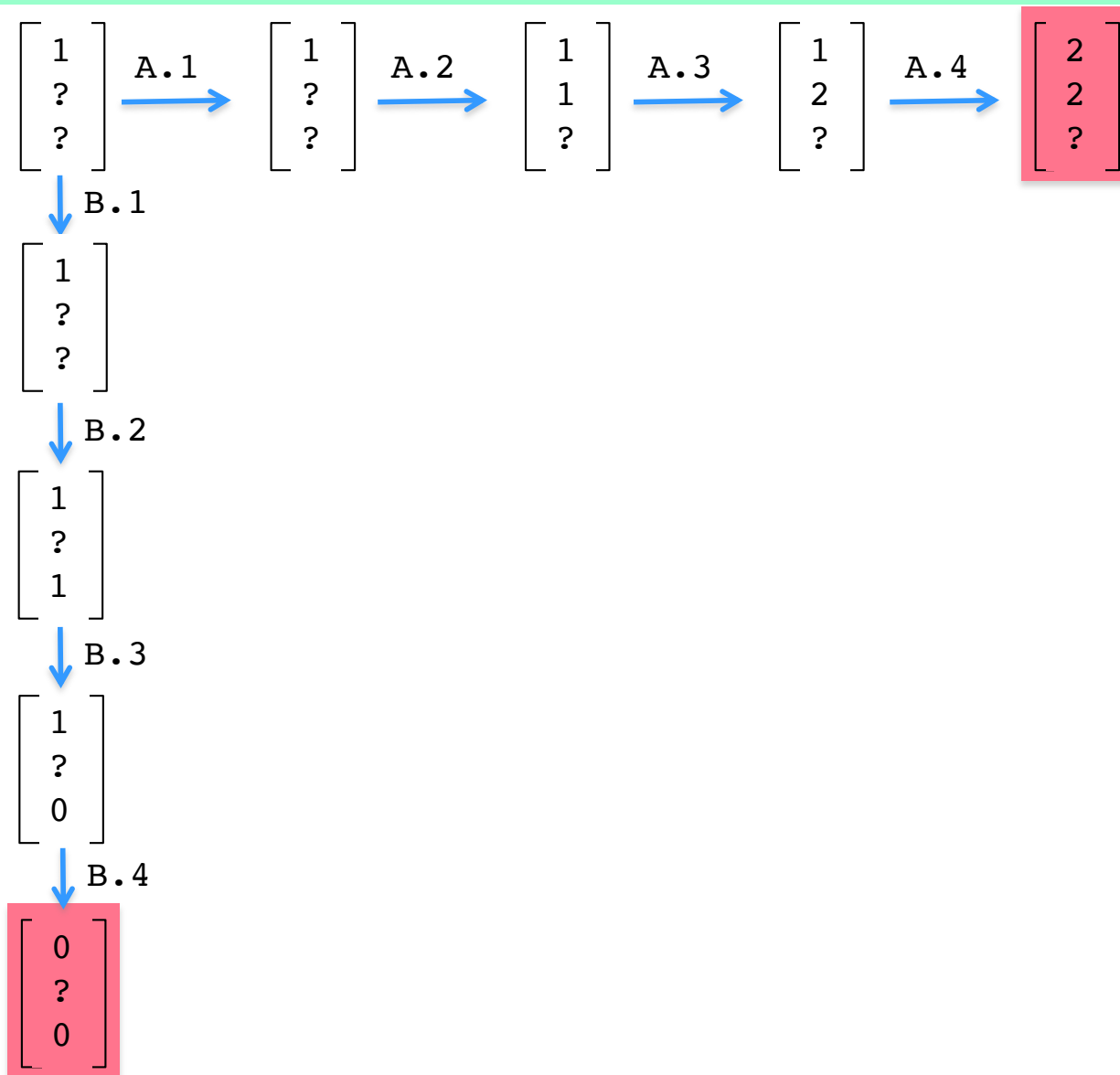
```
proctype A() {  
    byte tmp;  
    atomic {  
        (state == 1) -> tmp = state; tmp = tmp + 1; state = tmp  
    }  
}
```

```
proctype B() {  
    byte tmp;  
    atomic {  
        (state == 1) -> tmp = state; tmp = tmp - 1; state = tmp  
    }  
}
```

```
init {  
    run A();  
    run B()  
}
```

Une instruction bloquante dans une
séquence atomique « casse » l'atomicité

Graphe des états



Exclusion mutuelle (Peterson)

```
#define Aturn    0
#define Bturn    1

bit dem[2], prio;
```

```
active proctype A() {
    do
        :: dem[Aturn] = true;
           prio = Bturn;
           (dem[Bturn] == false || prio == Aturn);
           printf("A enters in CS\n");
           /* critical section */
           printf("A leaves CS\n");
           dem[Aturn] = false
    od
}
```

*option (au moins une
à l'intérieur de toute
structure de contrôle)*

```
active proctype B() {
    do
        :: dem[Bturn] = true;
           prio = Aturn;
           (dem[Aturn] == false || prio == Bturn);
           printf("B enters in CS\n");
           /* critical section */
           printf("B leaves CS\n");
           dem[Bturn] = false
    od
}
```

Emission / réception sur un canal

```
mtype = {T1, T2, ACK}  
chan can_AB = [1] of {mtype, int};
```

```
active proctype A() {  
    int x;  
    mtype t;  
    can_AB ? t(x);  
    printf("recu : %d(%d)\n", t, x);  
    can_AB ! ACK,45  
}
```

Déclaration d'un canal :
capacité et type de données
véhiculées

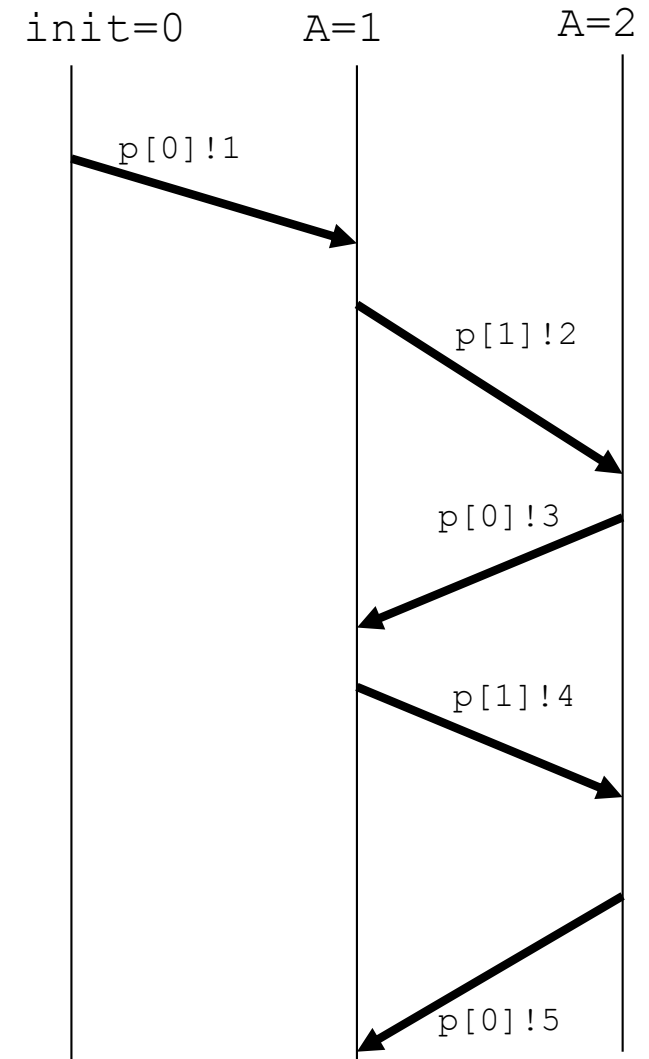
lecture sur le canal :
Le 1^{er} champ du message est
stocké dans t, le 2^{ème} dans x

```
active proctype B() {  
    int y;  
    can_AB ! T1(123);  
    can_AB ? ACK(y);  
    printf("y = %d\n", y)  
}
```

envoi sur le canal
du message (T1, 123)

Message Sequence Charts

```
proctype A(chan in, out) {  
  byte x;  
  do  
    :: in ? x -> out ! x+1  
  od  
}  
  
init{  
  chan p [2] = [1] of {byte};  
  
  atomic {  
    p[0]!1;  
    run A(p[0],p[1]);  
    run A(p[1],p[0]);  
  }  
}
```



Communications synchrones

```
#define msgtype 33
```

```
chan name = [0] of { byte, byte };
```

```
proctype A() {  
    name!msgtype, 124;  
    name!msgtype, 121  
}
```

```
proctype B() {  
    byte state;  
    name?msgtype, state  
}
```

```
init{  
    atomic {  
        run A();  
        run B()  
    }  
}
```

Canal de synchronisation (capacité nulle).

Une écriture ne peut être réalisée que simultanément à une lecture

cette instruction est bloquante :
pas de lecture en correspondance

Condition et indéterminisme

```
#define a 1
#define b 2

chan ch[2] = [1] of {byte};

proctype A(){
    ch[0]!a
}

proctype B(){
    ch[1]!b
}

proctype C(){
    byte x;
    if
        :: ch[0]?x
        :: ch[1]?x
    fi;
    printf("%d",x);
}
```

```
init {
    atomic {
        run A();
        run B();
        run C()
    }
}
```

L'entrée (::) exécutée est choisie de façon indéterministe parmi les instructions « franchissables »

Le programme affiche soit 1 soit 2

`if` est bloquant si aucune entrée exécutable
➤ la dernière entrée peut être conditionnée par le mot clé `else`

Répétition et indéterminisme

```
byte count1 = 1, count2 = 1;

proctype counter1() {
    do
        :: count1 = count1 + 1
        :: count1 = count1 - 1
        :: (count1 == 0) -> break;
    od
}

proctype counter2() {
    do
        :: (count2 != 0) ->
            if
                :: count2 = count2 + 1
                :: count2 = count2 - 1
            fi
        :: (count2 == 0) -> break
    od
}
```

Même principe que pour la structure conditionnelle

- Le premier compteur peut passer par la valeur 0 sans pour autant s'arrêter
- Le second compteur s'arrête systématiquement dès qu'il atteint la valeur 0

```
init {
    atomic {
        run counter1();
        run counter2();
    }
}
```

Canaux de communication (suite)

- Lecture sur un canal

`c?ACK` exécute la réception si le 1^{er} message dans `c` est ACK

`c?<ACK, x>` affecte à `x` la valeur `n` si le 1^{er} message dans `c` est `(ACK, n)`

`c?[ACK]` renvoie `true` si le 1^{er} message dans `c` est ACK

Les deux dernières instructions ne modifient pas l'état de `c`.

- Opérateur `len`

`len(<nom de canal>)`

Rend le nombre de messages présents dans le canal

- Opérateurs `empty` / `nempty`

`empty(<nom de canal>)` / `nempty(<nom de canal>)`

Rend vrai (resp. faux) si le canal est vide

Canaux non FIFO ou non fiables

- Pas de lecture aléatoire d'un canal, mais on peut manipuler un message qui n'est pas en tête de liste :

`c??ACK` exécute la réception si `c` contient un message `ACK`

`c??<ACK, x>` affecte à `x` la valeur `n` du 1^{er} message `(ACK, n)` dans `c`

`c??[ACK]` renvoie `true` si `c` contient un message `ACK`

Les deux dernières instructions ne modifient pas l'état de `c`.

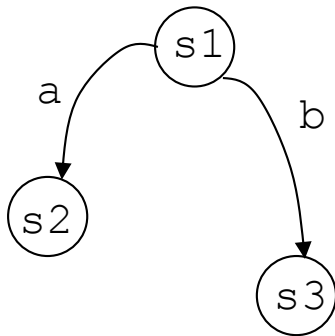
- On peut utiliser l'indéterminisme pour construire des canaux non fiables

```
do
  :: ch?x -> break;
  :: ch?_
od
```

`_` est une variable globale de type entier, prédéfinie, accessible uniquement en écriture.

Sauts, instructions d'échappement et nulle

- Les instructions peuvent être munies d'une étiquette.
- `goto label` permet de se brancher inconditionnellement à l'instruction d'étiquette `label`.
- `skip` est une instruction vide (pas d'autre effet que la modification du compteur ordinal du processus qui l'exécute) qui peut être associée à une étiquette.



```
proctype A() {  
    s1:  if  
        :: x ? a -> goto s2  
        :: x ? b -> goto s3  
        :: else  
        fi;  
    s2:  ...;  
        goto s1;  
    s3:  skip  
}
```

Déclaration et appels de sous-programme

- Pas de fonction ou de procédures, mais des « `inline` » : portion de code recopiée lors de l'appel (macro). Les paramètres ne sont pas typés.
- Les `inline` sont déclarés au même niveau que les processus. Un `inline` peut faire appel à un autre `inline` mais il ne peut y avoir de dépendance cyclique, ni d'appel récursif.

```
inline ma_procedure (x,y) {  
    code promela  
}
```

- Un `inline` ne renvoie pas de résultat à l'appelant
- Un `inline` n'a pas de variable locale : la portée d'une variable est soit le processus, soit le programme. Mais on peut déclarer une variable dans un `inline`.

Expansion des « inline »

- Possibilité de visualiser le code produit après expansion des inline :

spin -I source.pml

- Exemple :

```
#define N 2      /* Number of processes */
mtype = {msg1, msg2};
chan emission[N] = [3] of { mtype };
inline Broadcast (msg) {
    ...
}
active [2] proctype node() {
    mtype msg_recu;
    Broadcast (msg1);
    Broadcast (msg2);
    emission[_pid] ? msg_recu;
    emission[_pid] ? msg_recu;
}
```

Exemple

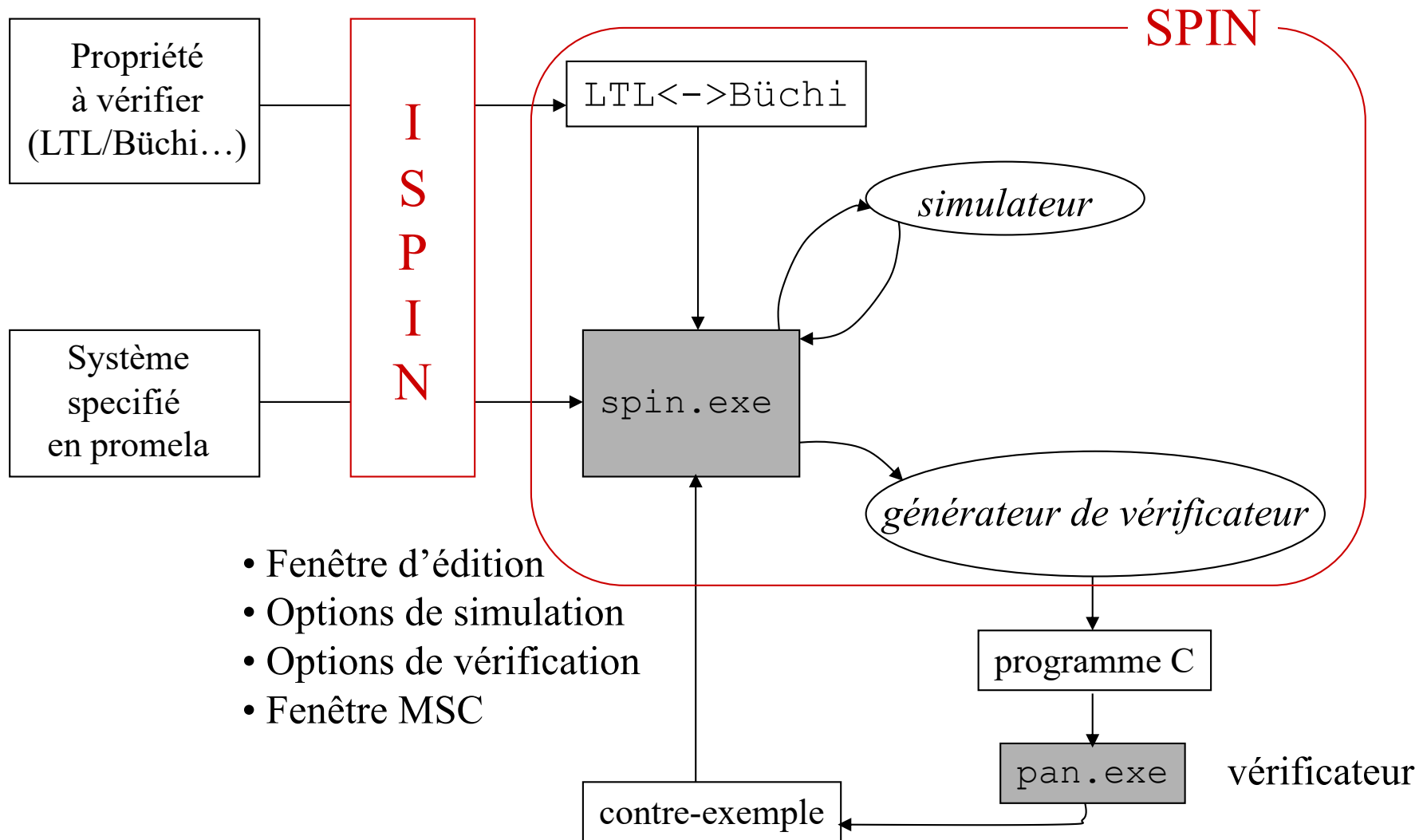
```
inline Broadcast (msg) {  
    byte i;  
    i=1;  
    do  
        :: i < N -> atomic {  
            emission[ (_pid+i) %N] !msg;  
        }  
        i++;  
        :: i == N -> break  
    od;  
}
```

`_pid` est l'identifiant unique du processus (entre 0 et 255).

```
proctype node()  
{  
    {  
        i = 1;  
        do  
            :: ((i<2));  
            atomic {  
                emission[ ((_pid+i)%2)] !msg1;  
            };  
            i = (i+1);  
            :: ((i==2));  
            goto :b0;  
        od;  
        :b0:  
    };  
    {  
        i = 1;  
        do  
            ...  
        od;  
    }  
}
```

Eviter les déclarations dans les `inline`

SPIN et iSpin



iSpin

Peterson_N.pml

Spin Version 6.5.2 -- 15 February 2024 :: iSpin Version 1.1.5 -- 28 May 2021

Edit/View Simulate / Replay Verification Swarm Run <Help> Save Session Restore Session <Quit>

Open... ReOpen Save Save As... Syntax Check Redundancy Check Symbol Table Find:

```
1 /* Peterson's algorithm for N processes - see Lynch's book, p. 284 */
2
3 #define N 3 /* nr of processes */
4
5 byte dernier[N], niveau[N];
6 byte ncrit; /* auxiliary var to check mutual exclusion */
7
8 active [N] proctype user() {
9   byte j, k;
10  /* the _pid's are: 0 .. N-1 */
11  again:
12    k = 0; /* count max N-1 rounds of competition */
13    do
14      :: k < N-1 ->
15      ask_SC :
16        niveau[_pid] = k;
17        dernier[k] = _pid;
18
19        j = 0; /* for all j != _pid */
20        do
21          :: j == _pid ->
22            j++;
23          :: j != _pid ->
24            if
25              :: j < N ->
```

Automata View zoom in zoom out

Spin Version 6.5.2 -- 15 February 2024
iSpin Version 1.1.5 -- 28 May 2021
TclTk Version 8.5/8.5
1 /Users/cdutheillet/Applications/Spin_6.5.2/Examples/_Mes_Exemples/Peterson_N.pml:1
2 syntax check
ltl enterSC: [] ((! ((user@ask_SC))) || (<= ((user@in_SC))))
3 spin -o3 -a Peterson_N.pml
ltl enterSC: [] ((! ((user@ask_SC))) || (<= ((user@in_SC))))
4 gcc -o pan pan.c
5 ./pan -D > dot.tmp
6 select p_user

Simulation (Simulate / Replay) avec ispin

- Type de simulation
 - aléatoire
 - interactive
 - guidée
- Paramétrage de l'exécution
 - gestion des canaux (blocage ou perte lors d'écriture dans canal plein)
 - borne sur la longueur maximale de la séquence
- Affichages
 - fenêtres d'affichage des variables et des canaux
 - fenêtre MSC
 - déroulement de l'exécution
- Ne prouve pas que le programme est correct. Peut prouver qu'il est faux.

Paramétrage d'une simulation

huang.pml

Spin Version 6.1.0 -- 4 May 2011 :: iSpin Version 1.0.5 -- 27 February 2011

Edit/View Simulate / Replay Verification Swarm Run <Help> Save Session Restore Session <Quit>

Mode

☒ Random, with seed: 1

☐ Interactive (for resolution of all nondeterminism)

☐ Guided, with trail: huang.pml.trail browse

initial steps skipped: 0

maximum number of steps: 10000

☒ Track Data Values (this can be slow)

A Full Channel

☒ blocks new messages

☐ loses new messages

☐ MSC+stmnt

MSC max text width 20

MSC update delay 25

Output Filtering (reg. exps.)

process ids:

queue ids:

var names:

tracked variable:

track scaling:

(Re)Run

Stop

Rewind

Step Forward

Step Backward

Background command executed:

spin -p -s -r -X -v -n234 -l -g -u 10000 huang.pml

Save in: msc.ps

```
1 #define N 3
2
3 mtype = {actif, passif};
4 mtype = {ack, agr, mes, term};
5
6 chan canaux[N] = [N] of {mtype, int, byte};
7 /* message = <type, val_horloge, emetteur> */
8
9 bool termine = 0;
10 /* positionne a vrai quand la terminaison est detectee */
11
12 /*****
13
14 inline max(h1, h2) { /* h1 = max(h1, h2) */
15   if
16   :: (h1 < h2) -> h1 = h2;
17   :: else -> skip
18   fi
19 }
20
21 /*****
22
23 inline desactivation() {
```

25 un_site:2

29 3!mes,0,1 un_site:3

31 2!mes,0,0

35 2!mes,0,0

42 2!mes,0,0

46 2!mes,0,0

52 1!ack,0,1

54 1!ack,0,1

63 2!ack,0,2

67 2!mes,0,0

73 2!mes,0,0

81 1!mes,0,2

86

[variable values, step 290]

```
:init:(0):i = 3
termine = 1
un_site(1):etat = passif
un_site(1):h = 3
un_site(1):hrec = 3
un_site(1):i = 3
un_site(1):last = 0
un_site(1):mes_tp = agr
un_site(1):nbagr = 2
un_site(1):sdr = 1
un_site(1):unack = 0
un_site(2):etat = passif
un_site(2):h = 3
un_site(2):hrec = 3
un_site(2):i = 3
un_site(2):last = 0
un_site(2):mes_tp = term
un_site(2):sdr = 0
un_site(2):unack = 0
un_site(3):etat = passif
un_site(3):h = 3
```

271: proc 3 (un_site) huang.pml:89 (state 84) [last = sdr]

272: proc 3 (un_site) huang.pml:90 (state 85) [h = hrec]

273: proc 3 (un_site) huang.pml:91 (state 86) [canaux[sdr][agr,h,id]

274: proc 1 (un_site) huang.pml:126 (state 106) [(nempty(canaux[id]))]

275: proc 2 (un_site) huang.pml:81 (state 93) [else]

276: proc 2 (un_site) huang.pml:87 (state 89) [(((hrec=h)&&(sdr>last)))]

277: proc 2 (un_site) huang.pml:89 (state 84) [last = sdr]

278: proc 2 (un_site) huang.pml:90 (state 85) [h = hrec]

279: proc 2 (un_site) huang.pml:91 (state 86) [canaux[sdr][agr,h,id]

280: proc 1 (un_site) huang.pml:139 (state 35) [canaux[id]?mes_tp,hrec,sdr]

281: proc 1 (un_site) huang.pml:109 (state 105) [((mes_tp==agr))]

282: proc 1 (un_site) huang.pml:99 (state 102) [nbagr = (nbagr+1)]

283: proc 1 (un_site) huang.pml:101 (state 100) [else]

284: proc 1 (un_site) huang.pml:104 (state 99) [(t1)]

285: proc 1 (un_site) huang.pml:126 (state 106) [(nempty(canaux[id]))]

286: proc 1 (un_site) huang.pml:139 (state 35) [canaux[id]?mes_tp,hrec,sdr]

287: proc 1 (un_site) huang.pml:109 (state 105) [((mes_tp==agr))]

288: proc 1 (un_site) huang.pml:99 (state 102) [nbagr = (nbagr+1)]

289: proc 1 (un_site) huang.pml:101 (state 100) [((nbagr==(3-1)))]

290: proc 1 (un_site) huang.pml:103 (state 97) [termine = 1]

timeout

#processes: 4

290: proc 3 (un_site) huang.pml:126 (state 106)

290: proc 2 (un_site) huang.pml:126 (state 106)

290: proc 1 (un_site) huang.pml:126 (state 106)

290: proc 0 (init) huang.pml:158 (state 11)

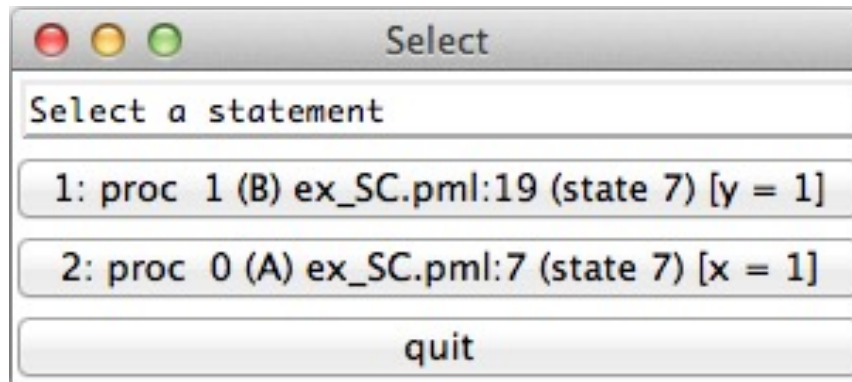
4 processes created

[queues, step 290]

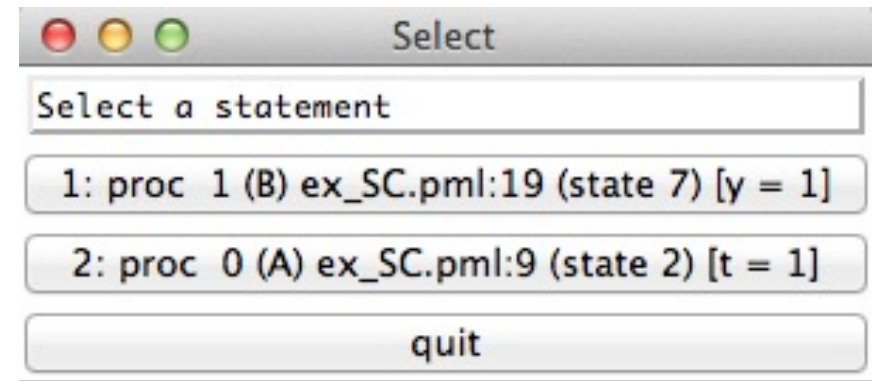
```
q 1 :: (canaux[0]):
q 2 :: (canaux[1]):
q 3 :: (canaux[2]):
```

Simulation interactive

```
0:   proc - (:root:) creates proc 0 (A)
0:   proc - (:root:) creates proc 1 (B)
```



```
0:   proc - (:root:) creates proc 0 (A)
0:   proc - (:root:) creates proc 1 (B)
Selected: 2
1:   proc 0 (A) ex_SC.pml:7 (state 7) [x = 1]
```



Simulation aléatoire

- Exécute le nombre d'étapes déterminé par la borne max.



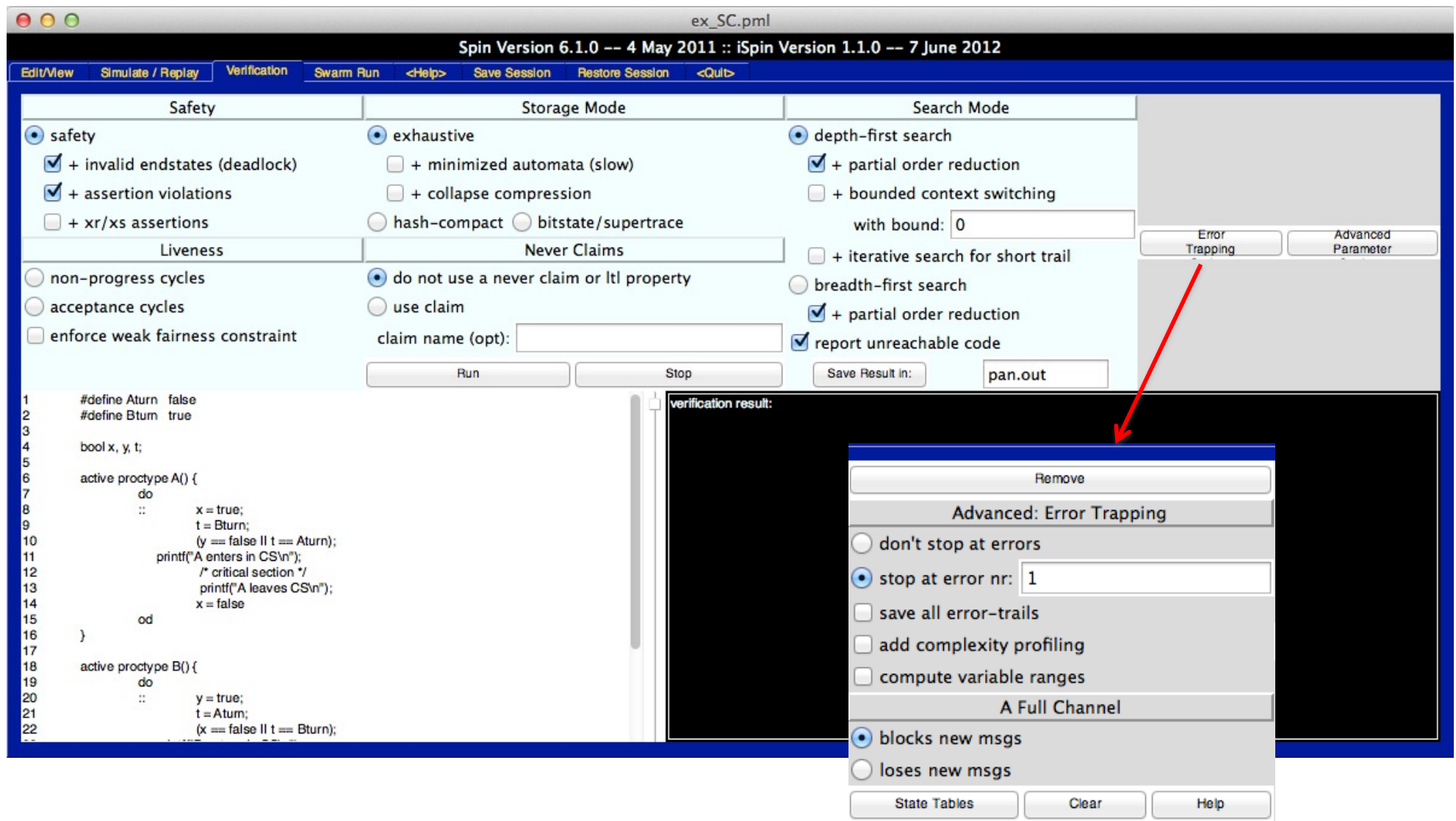
Si la borne est trop élevée, l'exécution est longue !

- On peut ensuite « rembobiner » (rewind) et revenir à une étape donnée
 - en cliquant sur la ligne correspondante dans le panneau d'exécution
 - en cliquant sur la boîte correspondante dans la fenêtre MSC
- A partir de ce point d'exécution, on peut ensuite avancer (step forward) ou reculer (step backward) pas à pas.

Vérification avec iSpin

- Permet de garantir qu'*aucune* exécution ne produira des situations indésirables :
 - Blocage
 - Plusieurs processus en section critique
 - Famine
- Classification des propriétés :
 - Basiques : terminaison, assertions
 - Sûreté : rien de mauvais ne se produira
 - Vivacité : quelque chose de bien finira par arriver

Paramétrage de la vérification



Exemple d'assertion

- `assert (condition);`
 - N'est jamais bloquant
 - Renvoie une erreur si `condition` est fausse
 - La valeur de `condition` n'est évaluée qu'au moment de l'exécution de l'instruction → définir l'instruction dans un processus séparé si on veut l'évaluer systématiquement.
- `xr c;`
 - Le processus qui inclut cette assertion affirme qu'il a un accès exclusif en lecture au canal `c`. Toute lecture par un autre processus sera considérée comme une erreur.
- `xs c;`
 - Accès exclusif en écriture au canal `c`.

Exemple

```
bool demande[2];  
bool tour;  
byte nb;
```

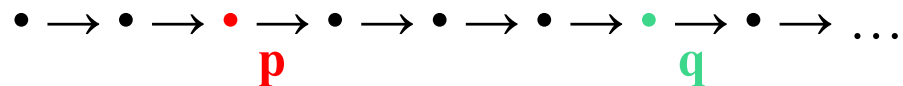
```
active [2] proctype P() {  
    demande[_pid] = 1;  
    do  
        :: (tour != _pid) ->  
            (!demande[1- _pid]);  
        tour = _pid;  
        :: (tour == _pid) ->  
            break;  
    od;  
    nb++;  
    skip;      /* SC */  
    assert(nb == 1);  
    nb--;  
    demande[_pid] = 0;  
}
```

Propriétés LTL (une introduction)

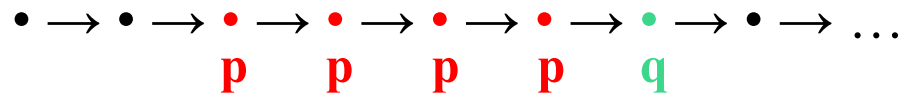
- LTL (*Linear Time Logic*) est une logique temporelle
- Ne prend pas en compte un temps quantifié, mais l'évolution du système
 - permet de définir des propriétés sur une séquence d'exécution
- Une formule est composée de propositions atomiques, opérateurs logiques, opérateurs temporels
- Des algorithmes efficaces permettent de valider une propriété LTL sur l'ensemble des exécutions du système

Exemples de propriétés

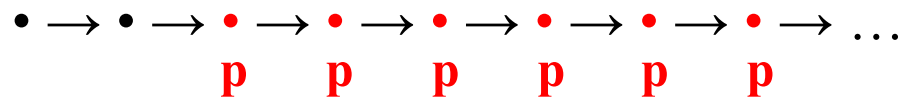
- Si p est vraie dans l'état courant, alors plus tard, q deviendra vraie



- Lorsque p devient vraie, elle le reste jusqu'à ce que q le devienne



- Lorsque p devient vraie, elle le reste indéfiniment



- ...

Formule LTL

- Propositions atomiques
 - Caractérisent un état ou un ensemble d'états, indépendamment de l'évolution du système
 - Expressions booléennes ($==$, $<$, $>$, $\&\&$, $\parallel$, ...) portant sur
 - Les variables
 - Les canaux
 - Les processus (opérateur $@$, variable $_pid$)
- Opérateurs logiques :
 - $\&\&$, $\parallel$, $!$, $\rightarrow$ (*implies*)
- Opérateurs temporels :
 - $[]$ (*always*), $\langle \rangle$ (*eventually*), U (*until*)

NB : eventually = se produira dans le futur

Propriétés temporelles classiques

spin ne fournit plus de « template » pour les propriétés 🥲

Construire une propriété est difficile. Heureusement,
beaucoup de propriétés relèvent de schémas classiques :

- **Invariance** : lorsque p devient vraie, elle le reste ensuite tout au long de l'exécution
- **Réponse** : chaque fois que p devient vraie, on passe ensuite par un état où q est vraie
- lorsque p devient vraie, elle le reste jusqu'à ce que q devienne vraie

Exemples

- s atteint l'état « terminé », et lorsqu'il l'atteint, il ne le quitte plus :
• → • → • → • → • → • → • → • → ...
(non terminé) *until (always terminé)*
- Si s atteint l'état terminé, il ne le quitte plus :
• → • → • → • → • → • → • → • → ...
ou • → • → • → • → • → • → • → • → ...
always (terminé implies always terminé)
- Chaque fois que s demande la section critique, il finit par l'obtenir
• → • → • → • → • → • → • → • → ...
always (demande implies (eventually servi))

Peterson pour 2 processus

```
bit dem[2], prio;
byte nb = 0;

active[2] proctype user() {
    pid moi = _pid, lui = 1 - _pid;

    do
        :: dem[moi] = true;
        ask_SC:
            prio = lui;
            (dem[lui] == false || prio == moi);
        in_SC:
            nb++;
            assert(nb == 1);
            nb--;
            dem[moi] = false
    od
}
```

Absence de famine

- Tout processus qui demande la section critique finira par l'obtenir (propriété de *réponse*).
- Quel que soit l'état dans lequel se trouve l'application, si cet état correspond à une demande de SC, alors il sera suivi dans le futur d'un état où le processus demandeur est en SC.

- Application à Peterson :

```
#define p (user[0]@ask_SC)
#define q (user[0]@in_SC)
always (p implies eventually q)
```


Vérification d'une formule LTL

- La formule doit être ajoutée au programme

- Ajout des macros :

- #define p (user[0]@ask_SC)**

- #define q (user[0]@in_SC)**

- Ajout de la formule :

- ltl abs_famine { [] (p -> <> q) }**

- ou :

- ltl abs_famine {**

- always (p implies eventually q) }**

- On coche ensuite l'option « use claim » lors du paramétrage de la vérification

Prise en compte de l'équité

- Spin examine *toutes* les séquences d'exécution possibles.
- Certaines ne correspondent pas à la réalité de l'environnement → introduction de la notion d'équité.
- **Equité faible (weak fairness)** : lorsque la prochaine action d'un processus est exécutable de manière permanente, elle finira par être exécutée.
- **Equité forte** : lorsque la prochaine action d'un processus est exécutable infiniment souvent, elle finira par être exécutée.

Exécution répartie et équité

- Non prise en compte de l'équité faible $\Rightarrow$ un processus qui n'est pas en panne ne s'exécute jamais
- Ajouter cette hypothèse à la vérification est conforme au contexte
- Une propriété fausse dans un contexte non équitable peut devenir vraie dans un contexte équitable
- L'équité ne s'applique qu'entre des processus différents
 - une action qui est toujours en compétition localement avec une autre action peut ne jamais être choisie

Conclusion

- Spin permet la vérification de propriétés :
 - exclusion mutuelle
 - absence de famine
 - ...
- La vérification explore tous les comportements possibles d'une application ($\neq$ simulation et $\neq$ exécution)
 - gourmande en mémoire : attention au choix de représentation !
- Faire les bonnes hypothèses sur le contexte peut éviter la construction de séquences non significatives
 - le contexte est-il équitable ?